SNOWFLAKE SQL PRACTICAL

Subject: Data Foundations

Platform: Snowflake Cloud Data Warehouse

Topic: Database Creation, Warehouse Management, Table Creation, CSV Data Loading and SQL Operations

Objective:

To understand and practise Snowflake SQL operations by creating a database, managing schemas and virtual warehouses, creating tables, preparing CSV file formats, uploading files to internal stages, and loading and verifying student academic performance data using SQL commands.

PART 1: DATABASE AND SESSION MANAGEMENT

Step 1: Check Current Session Information

Objective: To retrieve information about the current Snowflake session.

SQL Command:

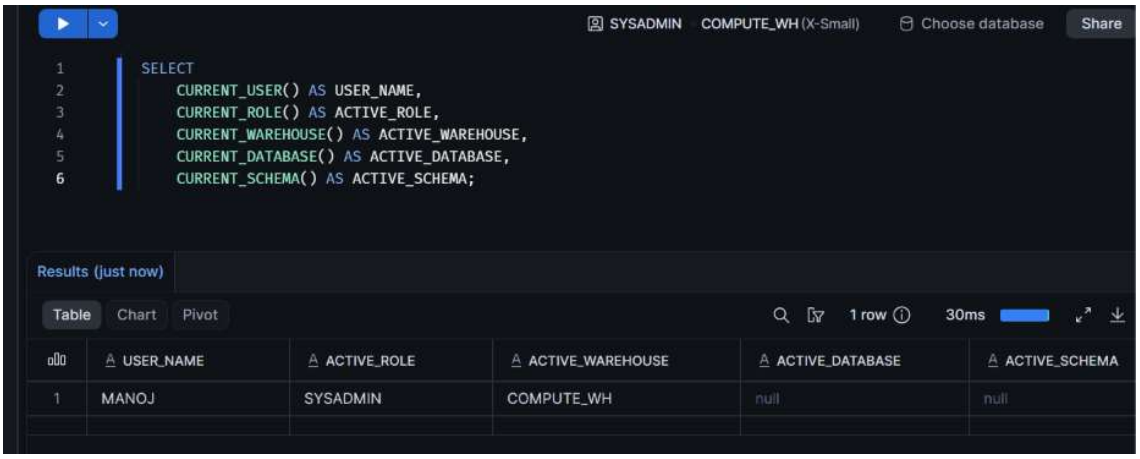
```
SELECT
  CURRENT_USER() AS USER_NAME,
  CURRENT_ROLE() AS ACTIVE_ROLE,
  CURRENT_WAREHOUSE() AS ACTIVE_WAREHOUSE,
  CURRENT_DATABASE() AS ACTIVE_DATABASE,
  CURRENT_SCHEMA() AS ACTIVE_SCHEMA;
```

Explanation:

- CURRENT_USER() displays the logged-in user.
- CURRENT_ROLE() displays the active role.
- CURRENT_WAREHOUSE() displays the active virtual warehouse.
- CURRENT_DATABASE() displays the selected database.
- CURRENT_SCHEMA() displays the selected schema.

Observation: The query displays the current Snowflake session details and helps identify the active working environment.

Screenshot:



Step 2: Create a Database

Objective:

To create a new database in Snowflake for storing student academic performance information.

SQL Command:

CREATE DATABASE IF NOT EXISTS STUDENT_DATA;

Explanation:

- CREATE DATABASE is used to create a new database in Snowflake.
- IF NOT EXISTS prevents an error if the database already exists.
- STUDENT_DATA is the name assigned to the database.

Verification Command:

SHOW DATABASES LIKE 'STUDENT_DATA';

Observation:

The STUDENT_DATA database was successfully created and verified using the SHOW DATABASES command.

Screenshot:





Step 3: Create a Schema

Objective:

To create a schema inside the STUDENT_DATA database for organizing student academic performance-related database objects.

SQL Command:

CREATE SCHEMA IF NOT EXISTS STUDENT_DATA.ACADEMICS;

Explanation:

- CREATE SCHEMA creates a new schema inside a database.
- IF NOT EXISTS prevents an error if the schema already exists.
- STUDENT_DATA is the database name.
- ACADEMICS is the schema name.

Select the Schema:

USE SCHEMA STUDENT_DATA.ACADEMICS;

This command selects ACADEMICS as the active schema for subsequent SQL operations.

Verification Commands:

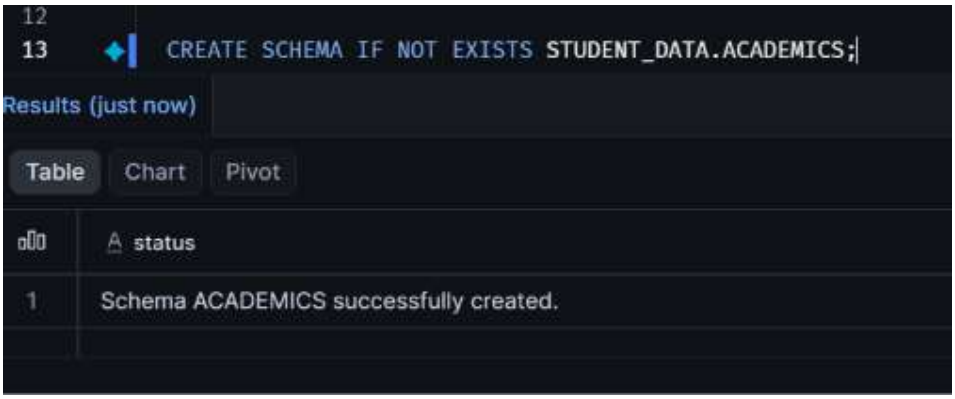
SHOW SCHEMAS IN DATABASE STUDENT_DATA;

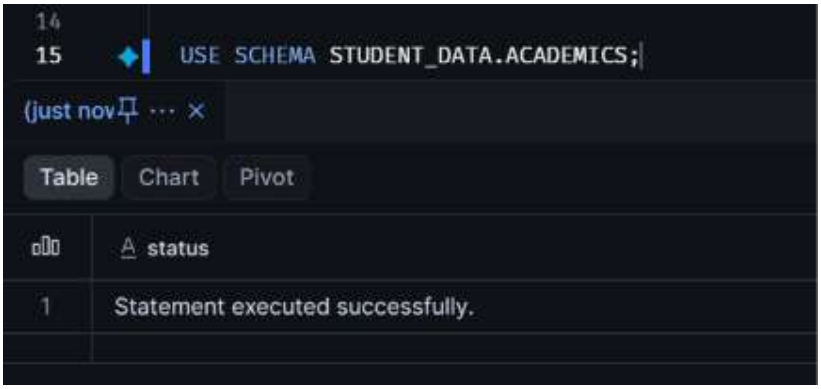
SELECT CURRENT_SCHEMA();

Observation:

The ACADEMICS schema was successfully created inside the STUDENT_DATA database and selected as the active schema for subsequent operations.

Screenshot:







Step 4: Create a Virtual Warehouse

Objective:

To create a virtual warehouse in Snowflake that provides the computing resources required to execute SQL queries and perform data loading operations.

SQL Command:

CREATE WAREHOUSE IF NOT EXISTS STUDENT_WH

WAREHOUSE_SIZE = 'XSMALL'

AUTO_SUSPEND = 60

AUTO_RESUME = TRUE

INITIALLY_SUSPENDED = TRUE;

Explanation:

- CREATE WAREHOUSE creates a virtual warehouse in Snowflake.
- IF NOT EXISTS prevents an error if the warehouse already exists.
- STUDENT_WH is the warehouse name.
- WAREHOUSE_SIZE = 'XSMALL' specifies the smallest standard warehouse size.
- AUTO_SUSPEND = 60 automatically suspends the warehouse after 60 seconds of inactivity.
- AUTO_RESUME = TRUE allows the warehouse to resume automatically when required.
- INITIALLY_SUSPENDED = TRUE creates the warehouse in a suspended state.

Select the Warehouse:

USE WAREHOUSE STUDENT_WH;

This command selects STUDENT_WH as the active virtual warehouse.

Verification Commands:

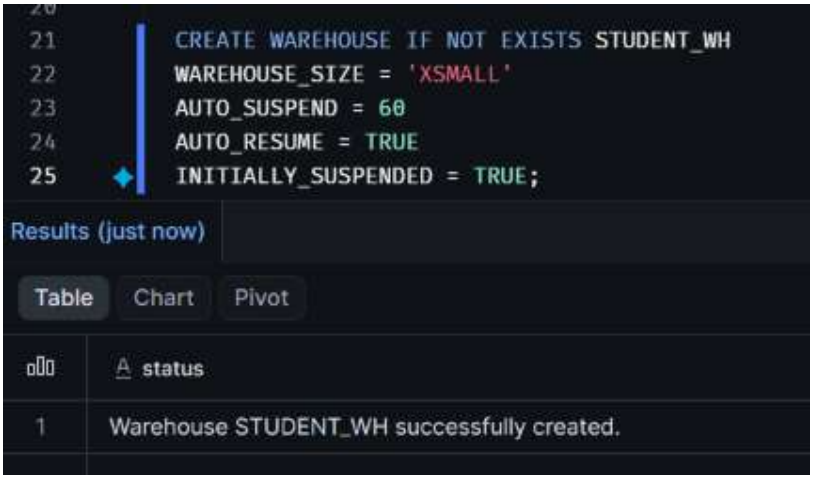
SHOW WAREHOUSES LIKE 'STUDENT_WH';

SELECT CURRENT_WAREHOUSE();

Observation:

The STUDENT_WH virtual warehouse was created with an XSMALL configuration and automatic suspension and resumption settings. It was selected as the active warehouse for subsequent SQL operations.

Screenshot:





Step 5: Create a Student Details Table

Objective:

To create a student details table inside the STUDENT_DATA database and ACADEMICS schema for storing student academic performance information.

SQL Command:

```
CREATE TABLE IF NOT EXISTS
STUDENT_DATA.ACADEMICS.STUDENT_DETAILS
(
   STUDENT_ID NUMBER(10,0),
   STUDENT_NAME VARCHAR(100),
   DEPARTMENT VARCHAR(50),
   YEAR_OF_STUDY NUMBER(2,0),
   MARKS NUMBER(5,2),
   CITY VARCHAR(100)
);
```

Explanation:

- CREATE TABLE creates a new table in Snowflake.
- IF NOT EXISTS prevents an error if the table already exists.
- STUDENT_ID stores the unique student identification number.
- STUDENT_NAME stores the student's name.
- DEPARTMENT stores the student's academic department.
- YEAR_OF_STUDY stores the current year of study.

- MARKS stores the student's academic marks.
- CITY stores the student's city.

Table Structure:

Column	Data Type	Purpose
STUDENT_ID	NUMBER(10,0)	Student identifier
STUDENT_NAME	VARCHAR(100)	Student name
DEPARTMENT	VARCHAR(50)	Academic department
YEAR_OF_STUDY	NUMBER(2,0)	Year of study
MARKS	NUMBER(5,2)	Academic marks
CITY	VARCHAR(100)	Student location

Verification Command:

```
SHOW TABLES LIKE 'STUDENT_DETAILS'

IN SCHEMA STUDENT_DATA.ACADEMICS;
```

Check Table Structure:

```
DESCRIBE TABLE STUDENT_DATA.ACADEMICS.STUDENT_DETAILS;
```

Observation:

The STUDENT_DETAILS table was successfully created inside the ACADEMICS schema with six columns and their respective data types.

Screenshot:



```
44
45  SHOW TABLES LIKE 'STUDENT_DETAILS'
    IN SCHEMA STUDENT_DATA.ACADEMICS;
```

Results (just now)

Table | Chart | Pivot — 1 row

	created_	name	database_name	schema_name	kind
1	2026-11-02 03:36:46	STUDENT_DETAILS	STUDENT_DATA	ACADEMICS	TABLE

```
46
47  DESCRIBE TABLE STUDENT_DATA.ACADEMICS.STUDENT_DETAILS;
```

Results (just now)

Table | Chart | Pivot — 6 rows — 40ms

	name	type	kind	null?	default	primary key	unique key
1	STUDENT_ID	NUMBER(10,0)	COLUMN	Y	null	N	N
2	STUDENT_NAME	VARCHAR(100)	COLUMN	Y	null	N	N
3	DEPARTMENT	VARCHAR(50)	COLUMN	Y	null	N	N
4	YEAR_OF_STUDY	NUMBER(2,0)	COLUMN	Y	null	N	N
5	MARKS	NUMBER(5,2)	COLUMN	Y	null	N	N
6	CITY	VARCHAR(100)	COLUMN	Y	null	N	N

Step 6: Create a CSV File Format

Objective:

To create a reusable CSV file format that defines how Snowflake reads and interprets student data during the data-loading process.

SQL Command:

```
CREATE FILE FORMAT IF NOT EXISTS

STUDENT_DATA.ACADEMICS.STUDENT_CSV_FORMAT

TYPE = CSV

SKIP_HEADER = 1

FIELD_OPTIONALLY_ENCLOSED_BY = '""'

EMPTY_FIELD_AS_NULL = TRUE;
```

Explanation:

- CREATE FILE FORMAT creates a named file format in Snowflake.
- STUDENT_CSV_FORMAT is the name of the file format.
- TYPE = CSV specifies that the input file is in CSV format.
- SKIP_HEADER = 1 skips the first row containing column names.

- FIELD_OPTIONALLY_ENCLOSED_BY = '"' handles values optionally enclosed in double quotes.
- EMPTY_FIELD_AS_NULL = TRUE converts empty fields into NULL values.

Verification Command:

SHOW FILE FORMATS

IN SCHEMA STUDENT_DATA.ACADEMICS;

View File Format Properties:

DESCRIBE FILE FORMAT

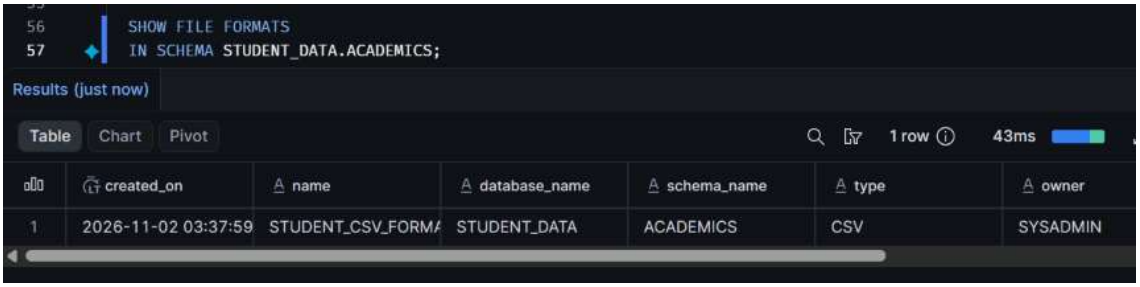
STUDENT_DATA.ACADEMICS.STUDENT_CSV_FORMAT;

Observation:

The STUDENT_CSV_FORMAT file format was successfully created with CSV parsing settings. It is configured to skip the header row, handle double-quoted fields, and interpret empty fields as NULL values.

Screenshot:





```
59  DESCRIBE FILE FORMAT
60  STUDENT_DATA.ACADEMICS.STUDENT_CSV_FORMAT;
```

Results (just now)

Table | Chart | Pivot — 24 rows — 27ms

	property	property_type	property_value	property_default
	BINARY_FORMAT 4.2% COMPRESSION 4.2% +22 more	String 54.2% Boolean 37.5% +2 more	true 20.8% AUTO 16.7% +12 more	true 20.8% AUTO 16.7% +11 more
1	TYPE	String	CSV	CSV
2	RECORD_DELIMITER	String	\n	\n
3	FIELD_DELIMITER	String	,	,
4	FILE_EXTENSION	String		
5	SKIP_HEADER	Integer	1	0
6	PARSE_HEADER	Boolean	false	false
7	DATE_FORMAT	String	AUTO	AUTO
8	TIME_FORMAT	String	AUTO	AUTO

Feedback

Step 7: Create an Internal Stage

Objective:

To create an internal stage in Snowflake for storing student CSV files before loading them into the STUDENT_DETAILS table.

SQL Command:

```
CREATE STAGE IF NOT EXISTS
STUDENT_DATA.ACADEMICS.STUDENT_STAGE
FILE_FORMAT = STUDENT_DATA.ACADEMICS.STUDENT_CSV_FORMAT;
```

Explanation:

- CREATE STAGE creates a named stage in Snowflake.
- IF NOT EXISTS prevents an error if the stage already exists.
- STUDENT_STAGE is the name of the internal stage.
- FILE_FORMAT specifies the CSV file format associated with the stage.
- STUDENT_CSV_FORMAT is the previously created file format.

Verification Command:

```
SHOW STAGES
IN SCHEMA STUDENT_DATA.ACADEMICS;
```

List Files Inside the Stage:

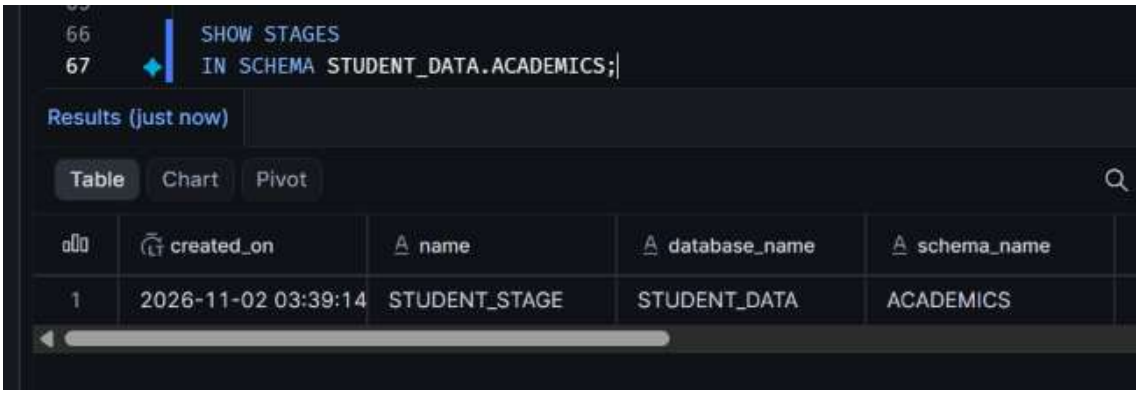
```
LIST @STUDENT_DATA.ACADEMICS.STUDENT_STAGE;
```

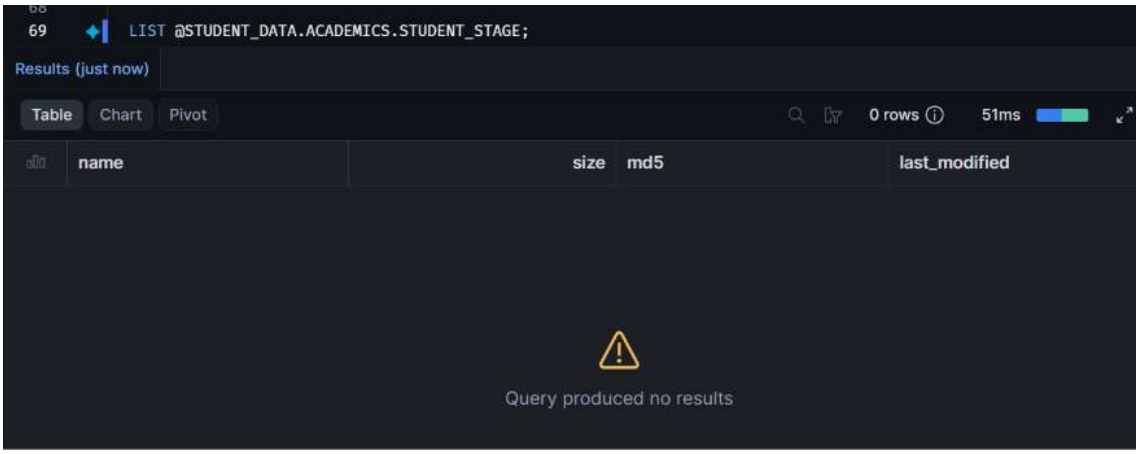
Observation:

The STUDENT_STAGE internal stage was successfully created and associated with the STUDENT_CSV_FORMAT file format. The stage is ready to receive student CSV files.

Screenshot:







Step 8: Prepare and Upload the CSV File

Objective:

To prepare a student academic performance CSV file and upload it into the Snowflake internal stage.

File Name: students.csv

Dataset Description:

The CSV file contains 10 student records with the following columns:

- STUDENT_ID
- STUDENT_NAME
- DEPARTMENT
- YEAR_OF_STUDY
- MARKS
- CITY

CSV Data:

STUDENT_ID,STUDENT_NAME,DEPARTMENT,YEAR_OF_STUDY,MARKS,CITY

3001,Manoj,Computer Science,4,92.50,Tirupati

3002,Rahul,Electronics,3,85.00,Chennai

3003,Priya,Computer Science,2,88.75,Bangalore

3004,Ananya,Mechanical,4,79.50,Hyderabad

3005,Kiran,Electronics,2,91.00,Mumbai

3006,Divya,Computer Science,3,95.25,Chennai

3007,Arjun,Civil,1,76.00,Pune

3008,Meera,Mechanical,3,84.50,Bangalore

3009,Sai,Computer Science,4,89.00,Hyderabad

3010,Nisha,Electronics,1,93.50,Mumbai

Data Preparation:

The student records were prepared in CSV format using Notepad and saved as students.csv.

Upload Procedure:

1. Open the Snowflake Snowsight interface.
2. Navigate to the STUDENT_DATA database.
3. Open the ACADEMICS schema.
4. Select the STUDENT_STAGE internal stage.
5. Use the Upload Files option.
6. Select the prepared students.csv file from the local system.
7. Upload the file to the internal stage.

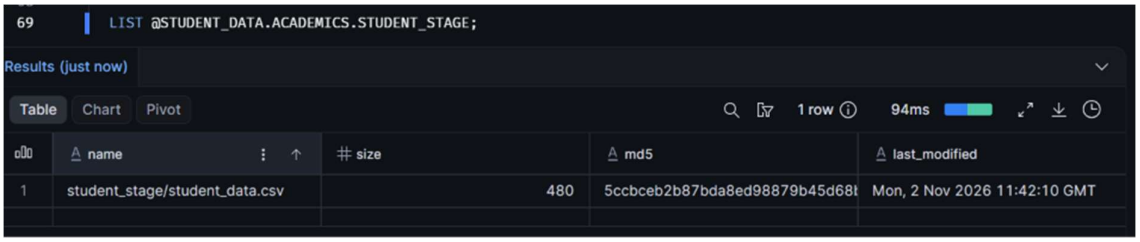
Verification Command:

LIST @STUDENT_DATA.ACADEMICS.STUDENT_STAGE;

Observation:

The student CSV file was uploaded to the STUDENT_STAGE internal stage and verified using the LIST command.

Screenshot:



Step 9: Load CSV Data Using COPY INTO

Objective:

To load student academic performance records from the CSV file uploaded to the internal stage into the STUDENT_DETAILS table using Snowflake's COPY INTO command.

SQL Command:

```
COPY INTO STUDENT_DATA.ACADEMICS.STUDENT_DETAILS
FROM @STUDENT_DATA.ACADEMICS.STUDENT_STAGE
FILE_FORMAT = (
  FORMAT_NAME =
  'STUDENT_DATA.ACADEMICS.STUDENT_CSV_FORMAT'
)
ON_ERROR = 'ABORT_STATEMENT';
```

Explanation:

1. COPY INTO loads data from a staged CSV file into a Snowflake table.
2. The destination table is STUDENT_DATA.ACADEMICS.STUDENT_DETAILS.
3. The FROM clause specifies the internal stage containing the CSV file.
4. FILE_FORMAT specifies the previously created CSV file format.
5. ON_ERROR = 'ABORT_STATEMENT' stops the loading operation if an error occurs.

Execution Result:

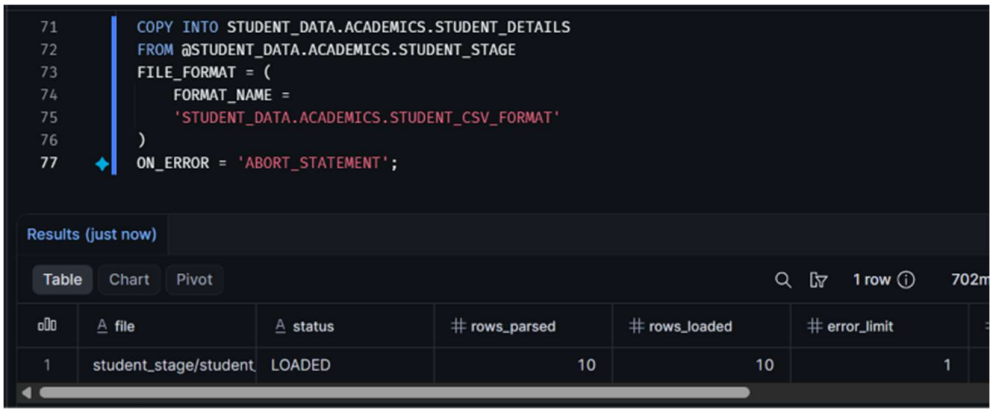
The execution result displays the following information:

- File name
- Loading status
- Number of rows parsed
- Number of rows loaded
- Errors encountered

Observation:

The student CSV file was processed using the COPY INTO command. The execution result indicates the number of student records successfully parsed and loaded into the STUDENT_DETAILS table.

Screenshot:



Step 10: Verify the Loaded Student Records

Objective:

To verify the student records loaded into the STUDENT_DETAILS table and confirm the total number of records.

Query 1: Display All Student Records

```
SELECT *

FROM STUDENT_DATA.ACADEMICS.STUDENT_DETAILS;
```

Query 2: Count Total Students

```
SELECT COUNT(*) AS TOTAL_STUDENTS

FROM STUDENT_DATA.ACADEMICS.STUDENT_DETAILS;
```

Query 3: Display the First Five Records

```
SELECT *

FROM STUDENT_DATA.ACADEMICS.STUDENT_DETAILS

LIMIT 5;
```

Explanation:

1. SELECT * retrieves all columns and records from the student details table.
2. COUNT(*) calculates the total number of records in the table.
3. LIMIT 5 restricts the query output to the first five records.

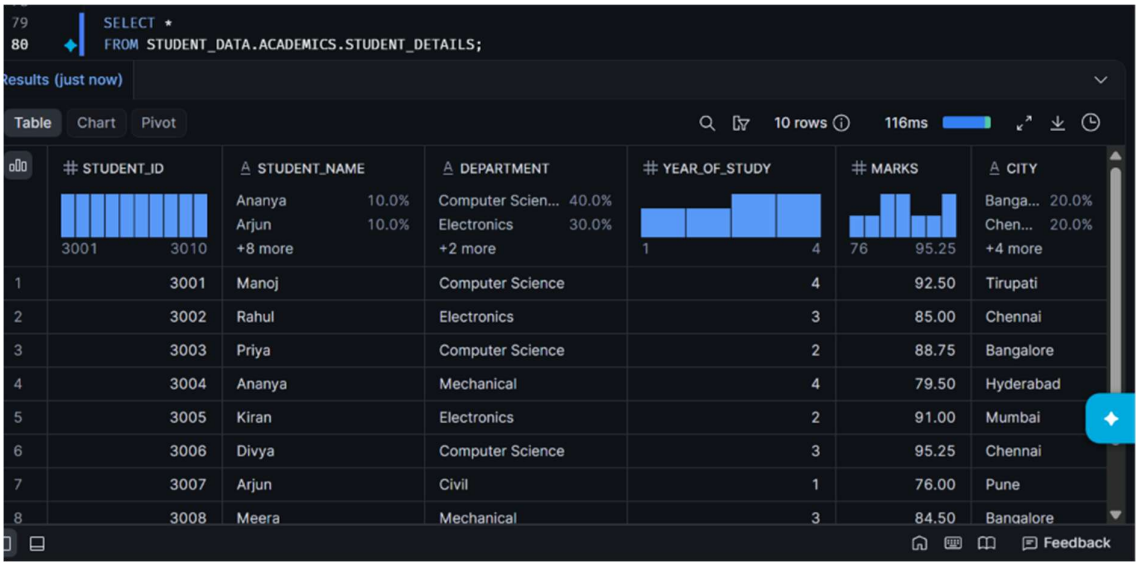
Observation:

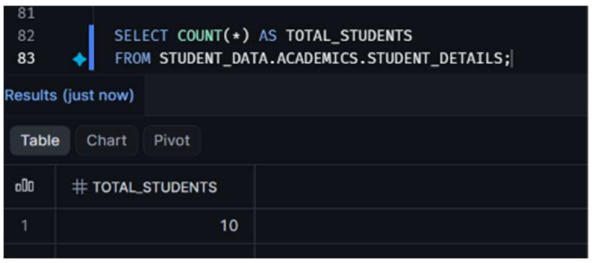
The student academic performance records were retrieved successfully from the STUDENT_DETAILS table. The COUNT(*) query confirms that 10 student records are present in the table, provided all ten records were loaded successfully.

Result:

The student CSV data was successfully loaded and verified in Snowflake.

Screenshots:





```
85  SELECT * FROM STUDENT_DATA.ACADEMICS.STUDENT_DETAILS
86  LIMIT 5;
```

Results (just now)

Table | Chart | Pivot — 5 rows — 55ms

	STUDENT_ID	STUDENT_NAME	DEPARTMENT	YEAR_OF_STUDY	MARKS	CITY
1	3001	Manoj	Computer Science	4	92.50	Tirupati
2	3002	Rahul	Electronics	3	85.00	Chennai
3	3003	Priya	Computer Science	2	88.75	Bangalore
4	3004	Ananya	Mechanical	4	79.50	Hyderabad
5	3005	Kiran	Electronics	2	91.00	Mumbai

PART 9: EXECUTION DOCUMENTATION AND RESULTS

Objective

To create and configure a Snowflake environment for managing student academic performance data, upload student records from a CSV file, load the data into a Snowflake table, and verify the successfully loaded records using SQL queries.

Steps Performed

The following operations were completed as part of this practical:

1. **Session Information:** Retrieved the current Snowflake user, active role, warehouse, database, and schema.
2. **Database Creation:** Created the STUDENT_DATA database.
3. **Schema Creation:** Created the ACADEMICS schema inside the database.
4. **Warehouse Creation:** Created the STUDENT_WH virtual warehouse and configured its auto-suspend and auto-resume properties.
5. **Table Creation:** Created the STUDENT_DETAILS table with columns for student ID, student name, department, year of study, marks, and city.
6. **File Format Creation:** Created the STUDENT_CSV_FORMAT file format to handle CSV data.
7. **Stage Creation:** Created the internal stage STUDENT_STAGE to store the CSV file.
8. **CSV File Upload:** Prepared the students.csv file containing student academic records and uploaded it to the internal stage.
9. **Data Loading:** Used the COPY INTO command to load the CSV records into the STUDENT_DETAILS table.
10. **Data Verification:** Executed SELECT queries to view the loaded records, count the total number of students, and retrieve the first five records.

Verification Results

The following SQL operations were used to verify the loaded data:

- SELECT * was used to retrieve all student records.

- COUNT(*) was used to verify the total number of records.
- LIMIT 5 was used to retrieve the first five records.

The dataset contained 10 student records. If the data loading operation completed successfully, the total record count should be 10.

Result

The Snowflake environment was successfully configured for the Student Academic Performance Management System. The database, schema, virtual warehouse, table, CSV file format, and internal stage were created.

The students.csv file was uploaded to the internal stage, and the COPY INTO command was used to load the student records into the STUDENT_DETAILS table. The loaded data was verified using SQL SELECT queries.

Conclusion: This practical demonstrated how Snowflake can be used to create a structured data environment, load data from CSV files, and perform basic SQL-based data verification.